# 1. PROJECT OVERVIEW

데이터 설명 및 문제 정의

정기예금 가입 여부를 예측하는 문제

데이터셋은 고객 정보, 마케팅 활동, 연락 정보 등 다양한 변수로 구성되어 있음

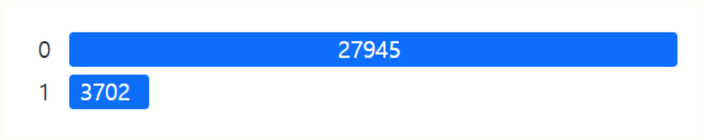
목표는 고객이 정기예금 가입할지 여부를 예측

| 변수명 | 타입 | 설명 | 카테고리 개수 | 변수명 | 타입 | 설명 | 카테고리 개수 |
|---|---|---|---|---|---|---|---|
| age | 수치형 | 나이 | - | contact | 수치형 | 최근 연락 한날 | 3 |
| job | 범주형 | 직업 | 12 | day | 수치형 | 최근 연락을 한 | - |
| marital | 범주형 | 학력 | 3 | month | 수치형 | 최근 연락을 한달 | 12 |
| education | 범주형 | 학력 | 4 | duration | 수치형 | 최근 연락 시 통화시간 | - |
| default | 범주형 | 신용채무 불이행 상태 | 2 | campaign | 수치형 | 이번 캠페인 관련 취한 연락 횟수 | - |
| balance | 수치형 | 연평균 잔액 | - | pdays | 수치형 | 지난 캠페인 관련하여 마지막 연락으로부터 경과한 일수 | - |
| housing | 범주형 | 주택대출여부 | 2 | previous | 수치형 | 이번 캠페인 이전에 해당 고객에게 취한 연락 횟수 | - |
| loan | 범주형 | 개인 대출여부 | 2 | poutcome | 범주형 | 이전 캠페인 참여여부 | 4 |

# 2. EDA

## 데이터의 분포 및 불균형

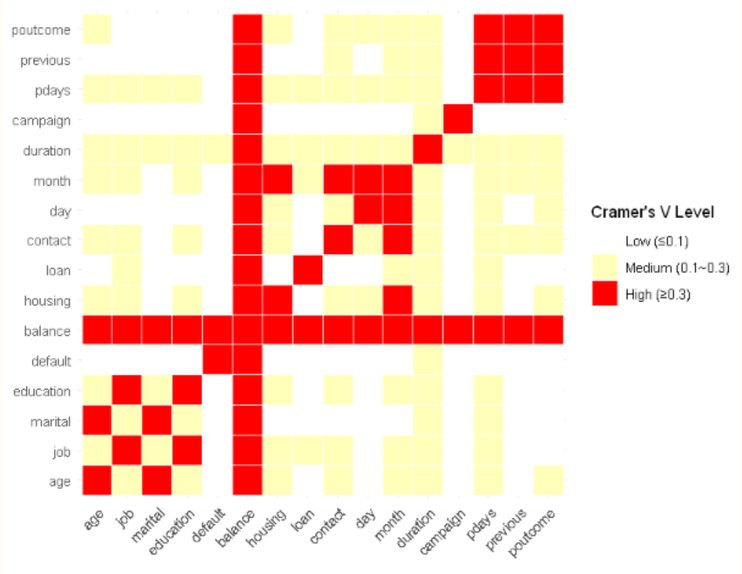
### 라벨값 데이터




라벨 값 불균형 (약 9:1 비율)

정기예금에 가입하지 않은 고객이 더 많으므로 추가적인 처리 필요

### 설명변수 간의 상관계수



# 2. EDA

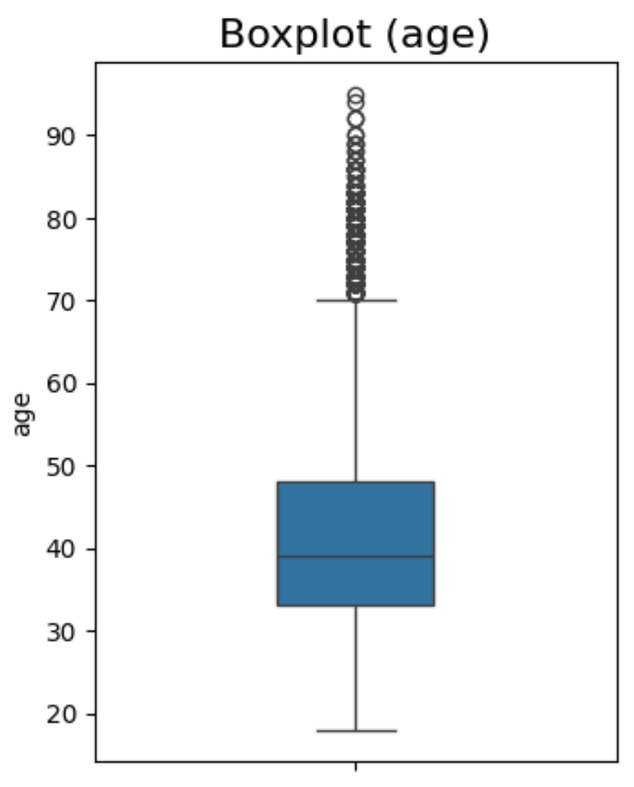
## 이상치 처리 방법

# 이상치 기준: (Q1 – 1.5*IQR) 보다 작거나 (Q3 + 1.5*IQR) 보다 큰 데이터
# IQR = Q3 - Q1

age: 젊은 연령대가 많아 통상적인 이상치 기준을 적용하면 고령층이 이상치로 검색됨
이를 모두 제거할 경우 고령층 데이터 학습에 방해가 될 것이라 판단
-> 이상치로 처리하지 않음

그 외 수치형 컬럼: 통상적인 이상치 기준으로는 이상치가 많이 검색됨
이상치를 SimpleImputer로 채워넣을 경우 데이터에 변형이 생기며,
해당하는 행을 모두 제거한 경우 역시 성능 하락을 야기하였음

이상치 기준 완화: ‘Duration’에만 로그 변환을 취함



# 3. FEATURE ENGINEERING

## 3.1 피처 엔지니어링의 필요성과 목적

**피쳐 엔지니어링**

1. 데이터가 모델 성능을 좌우

2. 모델이 학습하기 쉬운 형태로 데이터를 제공

3. 추가적인 정보를 제공해 모델의 한계를 보완

피처 엔지니어링은 데이터를 모델이 이해할 수 있도록 가공하고,
데이터에서 숨겨진 의미를 끌어내어 모델의 학습을 지원

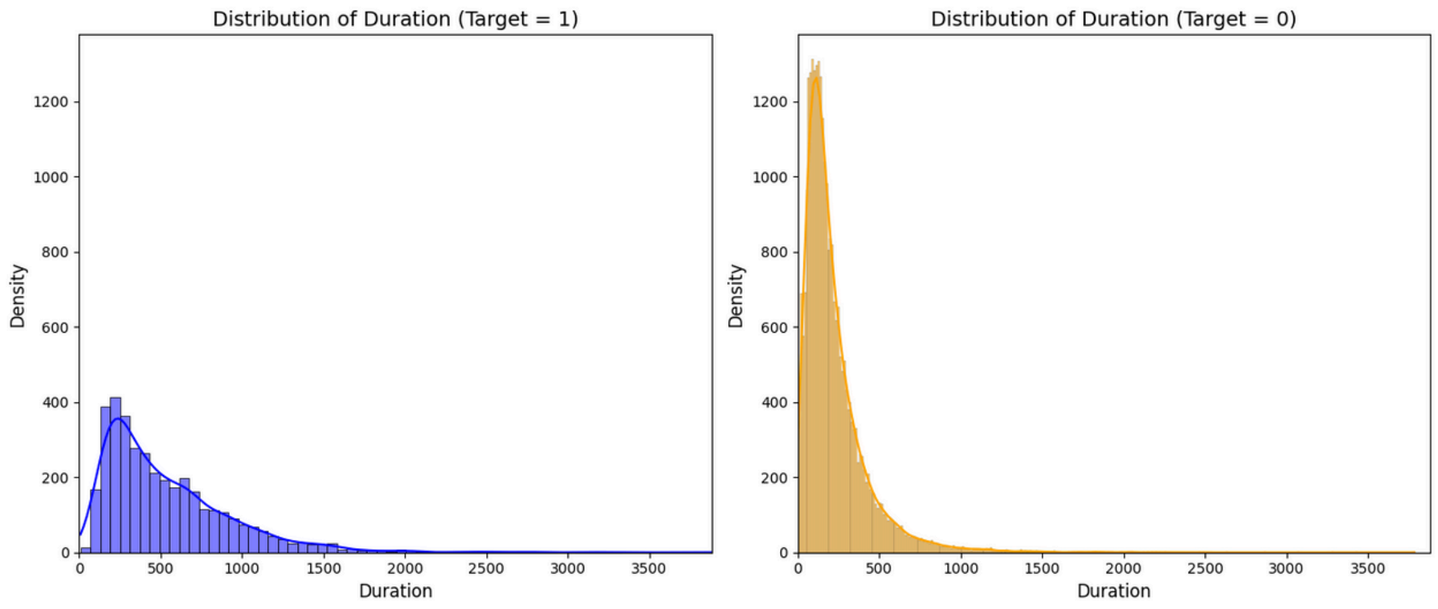
단순히 모델의 성능을 높이는 데 그치지 않고,
결과의 해석 가능성과 비즈니스 활용성도 동시에 증가

# 3. FEATURE ENGINEERING

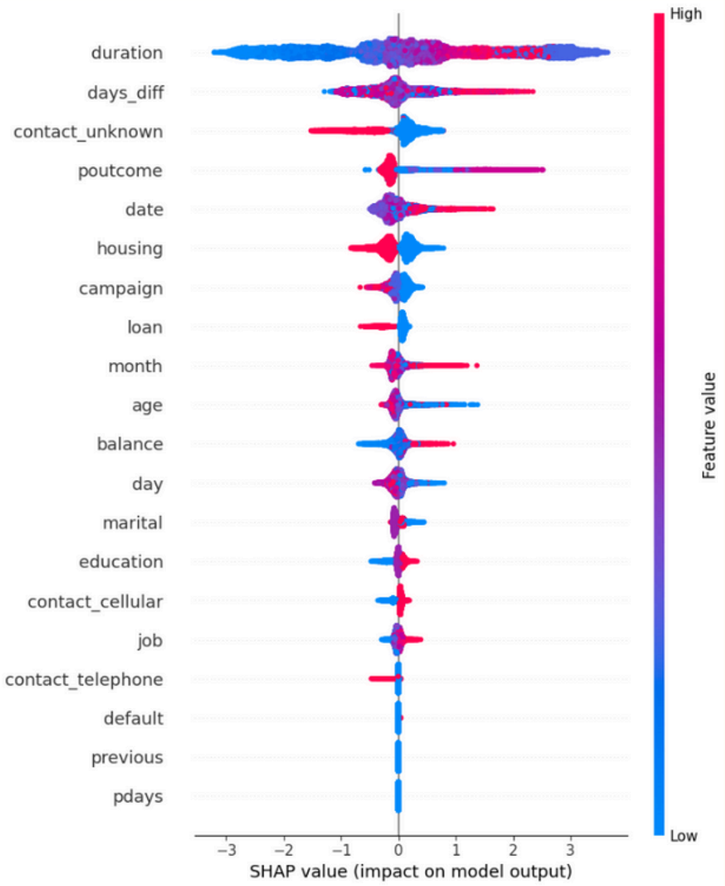
## 3.2 Duration 범주화

배경 : SHAP 확인 시 duration 변수의 기여도가 높음을 확인
-> Duration(상담 시간)이 모델 학습에 가장 큰 영향을 미침

duration의 분포 확인 시 0 ~ 1000 사이에 대부분의 값이 모여있음을 확인
-> 0에서 600 사이 구간을 세분화해 범주형으로 변환한 피쳐를 추가하여 모델 성능 개선




SHAP



# 3. FEATURE ENGINEERING

## 3.2 date, date_diff 생성

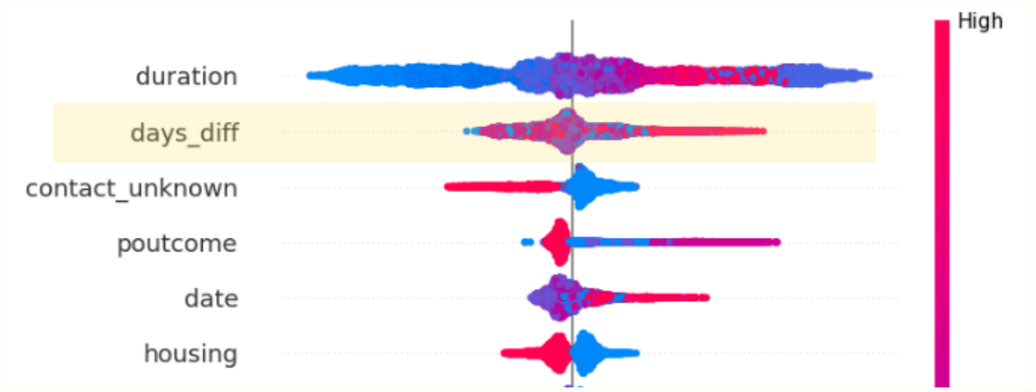
date 날짜 관련 데이터(day, month)를 단일 변수로 결합.

date_diff: 2024.01.01이라는 임의의 기준일을 정하고,
datetime으로 기준일과의 차이를 구해 int로 변환.

이유: pdays와 poutcome간 상관관계가 높음을 확인
-> month와 day를 결합해 경과일수를 구하면, 이번 캠페인에 대해서도
pdays와 poutcome간 상관관계와 유사한 관계를 만들 수 있을 것으로 기대

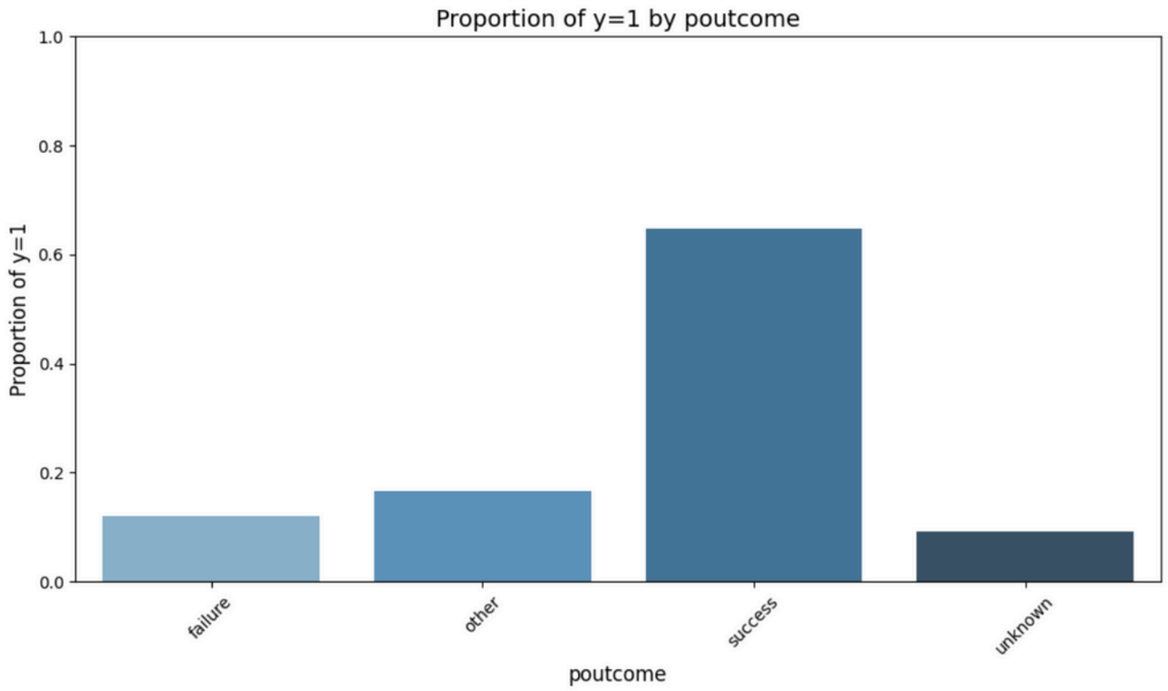
성과: 모델 해석력 향상 및 SHAP 분석 결과에서 높은 영향력 확인.

SHAP



# 3. FEATURE ENGINEERING

## 3.2 시도한 피처 엔지니어링 방법 : 주요 변수 생성 및 변환



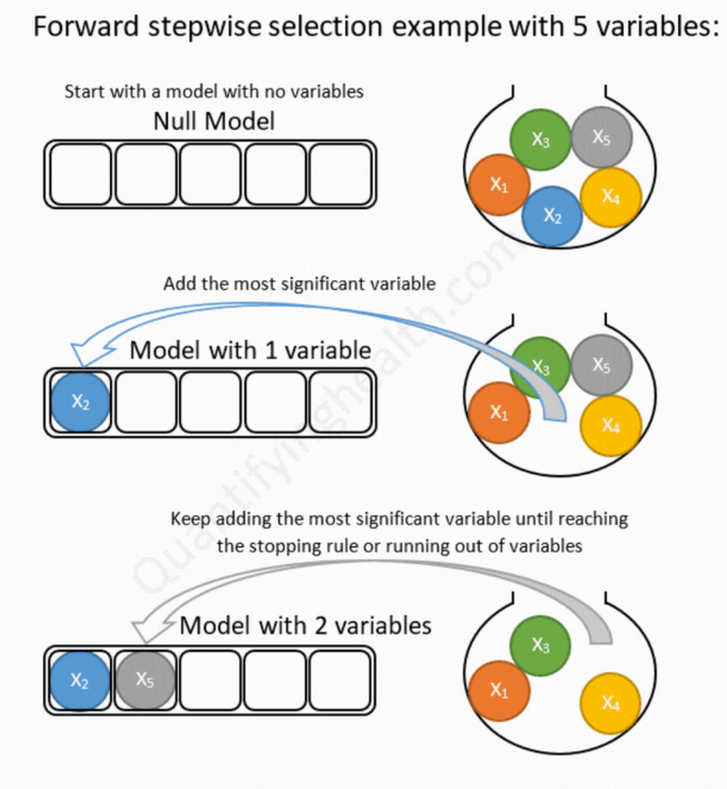

is_poutcome_success
이진 변수 생성

poutcome이 success 일 경우 y가 1일 확률 굉장히 높음

success면 1, 아니면 0으로 변수 생성

# 3. FEATURE ENGINEERING

## 3.2 시도한 피처 엔지니어링 방법 : FORWARD SELECTION




### 변수 제거

forward selection 활용하여 변수 조절

(1) 효율적인 피처 선택
(2) 해석 가능성 증가
(3) 과적합 방지
(4) 자동화 가능

# 3. FEATURE ENGINEERING

## 3.2 시도한 피처 엔지니어링 방법 : 주요 변수 생성 및 변환

### 변수 제거

성과 : previous, default 변수 제거 ->  0.62759

오버피팅 -> “포워드 셀렉션 기준 최고성능”과 비슷한 수준으로
변수를 줄여서 오버피팅을 예방하려고 하였으나

public score가 향상되지 않아서 채택하지 않았고,

차후 확인 결과 private score 또한 상승하지 않았음

더 상세한 분석을 위해 SHAP 확인

# 3. FEATURE ENGINEERING

## 3.3 피처 엔지니어링 결론

**모델 최고 성능 기록**

LGBM 모델에서 duration 범주화

+

date_diff

+

Job/Education 라벨 인코딩

↓

F1-score 0.66407 도달.

# 4.MODEL SELECTION

## 다양한 모델 테스트 결과 : 최적 모델 선택

| | |
|---|---|
| 기본적인 분류모델로 파악<br><br>다양한 모델 테스트 결과<br>(LGBM, SVM, XGB, RF 등)<br><br>전처리 및 인코더 처리한 XGB<br>(파이프라인 활용하여<br>범주형변수를 수치화하는 인코딩 전처리) | but.<br>고도화 전 단계인 LGBM 모델이 더 좋은 성능 보임<br>LGBM을 중심 모델로 선택<br><br>**y값 불균형 대응**<br>언더샘플링: 피쳐가 수치형이어야 사용 가능<br>LGBM 모델에 인코딩 -> 성능 하락<br>파라미터로 조정<br>('class_weight': ['balanced'] + [{0: 1, 1: i} for i in range(2, 9)]) |

# 4.MODEL SELECTION

## 상위 5개 모델 구성 설명 (submission_7)

### 1. 라이브러리 및 데이터 로드

데이터 분석과 머신러닝을 위한 필수 도구인 numpy, pandas 등 로드

```
import numpy as np
import os
import pandas as pd
import lightgbm as lgbm

from sklearn.metrics import f1_score
from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import RandomizedSearchCV
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import OneHotEncoder

import warnings
warnings.filterwarnings('ignore')
```

```
[ ] # 데이터 로드 및 데이터프레임 생성
    df_trn = pd.read_csv('data/train.csv')
    df_tst = pd.read_csv('data/test.csv')
```

# 4.MODEL SELECTION

submission_7

**2. 데이터 전처리**

**2-1. 날짜 처리 함수 정의**

day가 같으면 실제론 다른 날짜여도 같은 날짜로 학습할 위험이 있기 때문에 month와 day를 결합.

**date:** 연도에 대한 column이 없으므로, month와 date의 데이터가 동일 연도일 것이라 가정. 임의의 연도 2023과 함께 결합

**days_diff**: '최근 연락으로부터 경과한 일 수' 를 나타내는 변수 생성. 임의의 기준일 2024.01.01을 설정하고, 기준일과 date간 차이를 구함.

**pdays와 poutcome간의 상관관계가 유의미**했음을 확인. days_diff 변수 역시 pdays와 poutcome간의 관계처럼 이번 캠페인 예측 시 중요 정보가 될 것이라 기대

**'date' 형변환**: datetime 형의 'date' 변수를 LGBM 모델 학습 시 사용할 수 있도록 int 형변환

```python
from datetime import datetime

def preprocess_date(df):
    month_mapping = {
        'jan': 'January', 'feb': 'February', 'mar': 'March', 'apr': 'April',
        'may': 'May', 'jun': 'June', 'jul': 'July', 'aug': 'August',
        'sep': 'September', 'oct': 'October', 'nov': 'November', 'dec': 'December'
    }

    df['month'] = df['month'].map(month_mapping)
    df['date'] = '2023 ' + df['month'] + ' ' + df['day'].astype(str)
    df['date'] = pd.to_datetime(df['date'], format='%Y %B %d')
    current_date = datetime(2024, 1, 1)

    df['days_diff'] = (current_date - df["date"]).dt.days
    df.drop(['month', 'day'], axis=1, inplace=True)
    return df

df_trn = preprocess_date(df_trn)
df_tst = preprocess_date(df_tst)
df_trn['date'] = df_trn['date'].astype('int64')
df_tst['date'] = df_tst['date'].astype('int64')
```

# 4.MODEL SELECTION

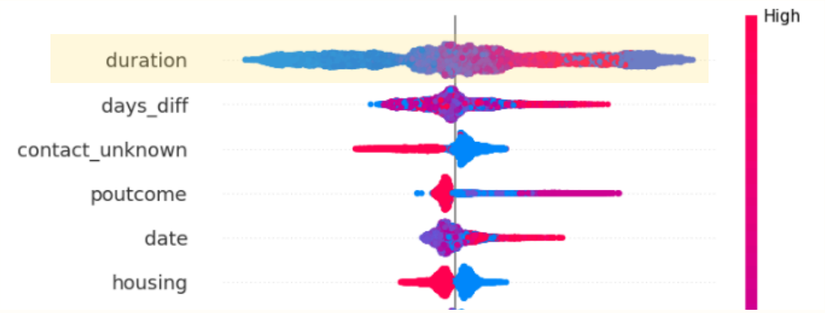
submission_7

**2. 데이터 전처리**

**2-2. 범주형 변수 변환 함수 정의**

이유: SHAP를 확인했을 때 duration 변수의 기여도가 높았음.
duration의 분포를 확인해 보면 0~500 사이 구간에
많은 데이터가 모여 있음을 확인.
따라서 연속형 데이터인 duration의 0에서 600 사이 구간을 세분화해
범주형으로 변환하여 모델 성능 개선.

SHAP




```python
def categorize_duration(value):
    if value == 0:
        return 'zero'
    elif value <= 100:
        return 'very_short'
    elif value <= 200:
        return 'short'
    elif value <= 400:
        return 'medium'
    elif value <= 600:
        return 'long'
    else:
        return 'very_long'
```

# 4.MODEL SELECTION

submission_7

### 3. 데이터 표준화 및 변환

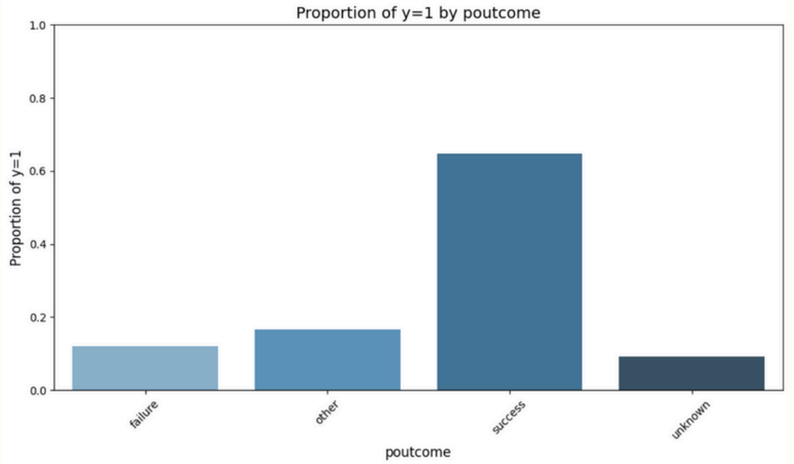
### 3-1. 로그 변환
분포가 치우친 데이터를 로그 변환하여 안정성 확보.

### 3-2. 'is_poutcome_success' 변수 생성
'poutcome' 변수의 범주값 별 타겟값 분포를 확인해 보았을 때,
'poutcome'이 'success'인 경우, y가 1일 확률이 높았음.

'poutcome'이 'success'인 경우 y값과 연관이 클 것이라 판단하고,
'poutcome'이 'success'인 경우 1, 그렇지 않은 경우 0으로 나타내는 변수 생성

```
[ ]  df_trn['duration_category'] = df_trn['duration'].apply(categorize_duration)
     df_trn['log_duration'] = np.log1p(df_trn['duration'])
```

```
[ ]  df_trn['is_poutcome_success'] = (df_trn['poutcome'] == 'success').astype(int)
```



# 4.MODEL SELECTION

submission_7

**4. 테스트 데이터 전처리**
Train 데이터와 동일 전처리 적용

```
df_tst['duration_category'] = df_tst['duration'].apply(categorize_duration)
df_tst['log_duration'] = np.log1p(df_tst['duration'])
df_tst['is_poutcome_success'] = (df_tst['poutcome'] == 'success').astype(int)

# 정렬 맞추기
df_tst = df_tst[df_trn.columns.drop('label')]
```

**5. 데이터 준비**

```
df_trn_y = df_trn['label']
```

**5-1. 타깃 변수 지정**
이유: 예측 대상과 입력 데이터를 구분하여 모델 준비.

```
for col in categorical_features:
    df_trn[col] = df_trn[col].astype('category')
    df_tst[col] = df_tst[col].astype('category')
```

**5-2. 불필요한 열 제거**
이유: 노이즈 제거로 모델 성능 최적화.
'ID', 'label': 모델 학습 시 제외해야 하는 열 제거.
'default', 'previous', 'pdays': SHAP 확인 시 최하위 3개 변수 제거.
inplace = True: 원본 객체를 직접 변경하도록 설정

```
df_trn.drop(columns=['ID','label','default','previous','pdays'], inplace=True)
tst_id = df_tst['ID']
df_tst.drop(columns=['ID','default','previous','pdays'], inplace=True)
```

# 4.MODEL SELECTION

submission_7

**6. 모델 생성 및 하이퍼파라미터 튜닝**

LightGBM 모델 생성.
RandomizedSearchCV시 사용할 param_grid 정의.
y값 불균형을 'class_weight' 파라미터로 조정
RandomizedSearchCV로 하이퍼파라미터 탐색

이유: 효율적인 탐색을 통해 성능 극대화.

```
model = lgbm.LGBMClassifier(verbose=-1,random_state=42)

param_grid = {
    'n_estimators': np.arange(100, 1000, 100),
    'learning_rate': np.arange(0.1, 0.9, 0.05),
    'class_weight': ['balanced'] + [{0: 1, 1: i} for i in range(2, 9)],
    'max_depth': np.arange(1, 8),
    'subsample': np.arange(0.5, 1.1, 0.1),
    'feature_fraction': [0.6, 0.7, 0.8, 0.9, 1.0],
    'max_bin': [255, 512, 1024],
    'random_state': [42]
}
random_search = RandomizedSearchCV(estimator=model,
                                   param_distributions=param_grid,
                                   scoring='f1',
                                   cv=10,
                                   refit=True,
                                   random_state=42,
                                   n_iter=100)
random_search.fit(df_trn, df_trn_y)
lgbm_best_model = random_search.best_estimator_
```

# 4.MODEL SELECTION

submission_7

7. 최적의 모델 선택

최적 파라미터 출력, 교차검증 점수 확인

```
[ ]  # subsample 근사값 출력문제 해결
     best_params = random_search.best_params_
     if 'subsample' in best_params:
         best_params['subsample'] = round(best_params['subsample'], 2)
```

```
     # Best parameters and score
     print(f"Best Parameters: {random_search.best_params_}")
     print(f"Best Cross-Validation Score: {random_search.best_score_:.2f}")
```

```
Best Parameters: {'subsample': 0.9, 'random_state': 42, 'n_estimators':
Best Cross-Validation Score: 0.63
```

+ 코드

```
[ ]  # 테스트 데이터에 대한 예측 수행
     predicted_labels = lgbm_best_model.predict(df_tst)
```

# 4.MODEL SELECTION

submission_5 ( submission_7 이전 모델 )

- date 열 삭제 (학습시 사용 안함)
- is_poutcome_success 피쳐엔지니어링 전

```python
from datetime import datetime

def preprocess_date(df):
    month_mapping = {
        'jan': 'January', 'feb': 'February', 'mar': 'March', 'apr': 'April',
        'may': 'May', 'jun': 'June', 'jul': 'July', 'aug': 'August',
        'sep': 'September', 'oct': 'October', 'nov': 'November', 'dec': 'December'
    }

    df['month'] = df['month'].map(month_mapping)
    df['date'] = '2023 ' + df['month'] + ' ' + df['day'].astype(str)
    df['date'] = pd.to_datetime(df['date'], format='%Y %B %d')
    current_date = datetime(2024, 1, 1)

    df['days_diff'] = (current_date - df["date"]).dt.days
    df.drop(['month', 'day', 'date'], axis=1, inplace=True)
    return df
```

```python
random_search = RandomizedSearchCV(estimator=model,
                                   param_distributions=param_grid,
                                   scoring='f1',
                                   cv=10,
                                   refit=True,
                                   random_state=42,
                                   n_iter=100)
random_search.fit(df_trn, df_trn_y)
lgbm_best_model = random_search.best_estimator_
```

```python
# subsample 근사값 출력문제 해결
best_params = random_search.best_params_
if 'subsample' in best_params:
    best_params['subsample'] = round(best_params['subsample'], 2)
```

# 4.MODEL SELECTION

submission_6 ( submission_7 이전 모델 )

- is_poutcome_success 열 없음
- date를 int로 변환하여 모델 학습에 사용

```
    def preprocess_date(df):
        month_mapping = {
            'jan': 'January', 'feb': 'February', 'mar': 'March', 'apr': 'April',
            'may': 'May', 'jun': 'June', 'jul': 'July', 'aug': 'August',
            'sep': 'September', 'oct': 'October', 'nov': 'November', 'dec': 'December'
        }

        df['month'] = df['month'].map(month_mapping)
        df['date'] = '2023 ' + df['month'] + ' ' + df['day'].astype(str)
        df['date'] = pd.to_datetime(df['date'], format='%Y %B %d')
        current_date = datetime(2024, 1, 1)

        df['days_diff'] = (current_date - df["date"]).dt.days
        df.drop(['month', 'day'], axis=1, inplace=True)
        return df

    df_trn = preprocess_date(df_trn)
    df_tst = preprocess_date(df_tst)
```

```
[ ]  random_search = RandomizedSearchCV(estimator=model,
                                        param_distributions=param_grid,
                                        scoring='f1',
                                        cv=10,
                                        refit=True,
                                        random_state=42,
                                        n_iter=100)
     random_search.fit(df_trn, df_trn_y)
     lgbm_best_model = random_search.best_estimator_
```

# 4.MODEL SELECTION

submission_11 ( submission_7 이후 모델 )

- duration_category에 K-Fold를 활용한 Target Encoding 추가 - 과적합 방지

```
[ ]  from sklearn.model_selection import KFold
     # Global mean 계산
     global_mean = df_trn['label'].mean()

     # KFold 설정
     kf = KFold(n_splits=5, shuffle=True, random_state=42)

     # 학습 데이터의 인코딩 값을 저장할 열 추가
     df_trn['encoded_label'] = 0

     # K-Fold를 활용한 타겟 인코딩
     for train_index, val_index in kf.split(df_trn):
         train_fold = df_trn.iloc[train_index]
         val_fold = df_trn.iloc[val_index]

         # Train 폴드에서 duration_category별 평균 계산
         duration_category_means = train_fold.groupby('duration_category')['label'].mean().to_dict()

         # Validation 폴드에 해당 평균값 매핑
         df_trn.loc[val_index, 'encoded_label'] = df_trn.loc[val_index, 'duration_category'].map(duration_category_means).fillna(global_mean)

     # 전체 학습 데이터로 최종 카테고리 평균 계산
     final_duration_category_means = df_trn.groupby('duration_category')['label'].mean().to_dict()

     # 테스트 데이터에 최종 카테고리 평균 매핑
     df_tst['encoded_label'] = df_tst['duration_category'].map(final_duration_category_means).fillna(global_mean)
```

# 4.MODEL SELECTION

submission_12 ( submission_7 이후 모델 )

- similar_months: 타깃값 1의 비율이 비교적 높은 월 'sep', 'mar', 'oct', 'dec'을 그룹화.해당 월이면 'similar', 그렇지 않으면 'other'로 그룹화

```python
from datetime import datetime

def preprocess_date(df):
    month_mapping = {
        'jan': 'January', 'feb': 'February', 'mar': 'March', 'apr': 'April',
        'may': 'May', 'jun': 'June', 'jul': 'July', 'aug': 'August',
        'sep': 'September', 'oct': 'October', 'nov': 'November', 'dec': 'December'
    }

    df['month'] = df['month'].map(month_mapping)
    df['date'] = '2023 ' + df['month'] + ' ' + df['day'].astype(str)
    df['date'] = pd.to_datetime(df['date'], format='%Y %B %d')
    current_date = datetime(2024, 1, 1)

    df['days_diff'] = (current_date - df["date"]).dt.days
    df.drop(['month', 'day'], axis=1, inplace=True)
    return df

similar_months = ['sep', 'mar', 'oct', 'dec']
df_trn['month_group'] = df_trn['month'].apply(lambda x: 'similar' if x in similar_months else 'ot

df_trn = preprocess_date(df_trn)

df_trn['date'] = df_trn['date'].astype('int64')
```

# 5.ENSENBLE STRATEGY

최종 제출 모델 설명 (binary_ensemble_result)

### 하드보팅 앙상블

단일모델 성능이 가장 좋았던
상위 LGBM 5개로 하드보팅 앙상블

### 모델 선택 이유

기본모델 기준: cat이랑 xgb rf 모델의 성능이 낮음
동일서열로 앙상블, LGBM에 가중치 부여해서 앙상블한 것보다
LGBM끼리만 앙상블한 것이 성능 우수

# 6.RESULT

binary_ensemble_result

public 0.66407점
private 0.64703점

# 7.CONCLUSION

## 프로젝트 요약 및 성과 & 향후 추가 연구 방향

### 요약 및 성과

- EDA와 피처 엔지니어링으로 데이터를 최적화하고 LGBM 모델로 성능을 평가
- K-Fold Target Encoding을 통해 과적합 방지
- 피쳐 생성으로 모델 해석력을 높임
- 앙상블을 통해 성능 향상

### 추가 연구 방향

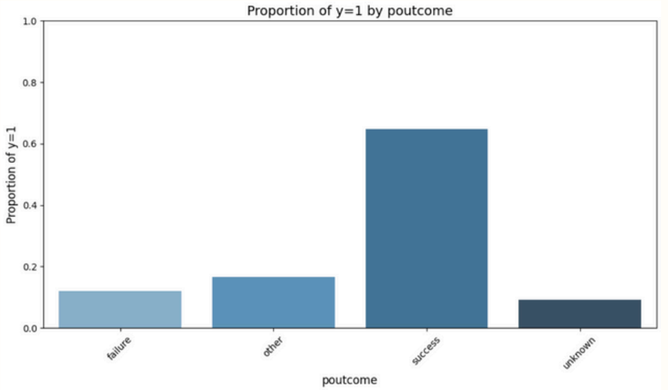
- 앙상블 시 다양한 모델을 사용해 모델 성능을 추가 분석할 필요가 있음
- 향후 스태킹, 배깅 등 고급 앙상블 기법을 적용해 개선 가능성을 탐색

# 7.CONCLUSION

## 인사이트

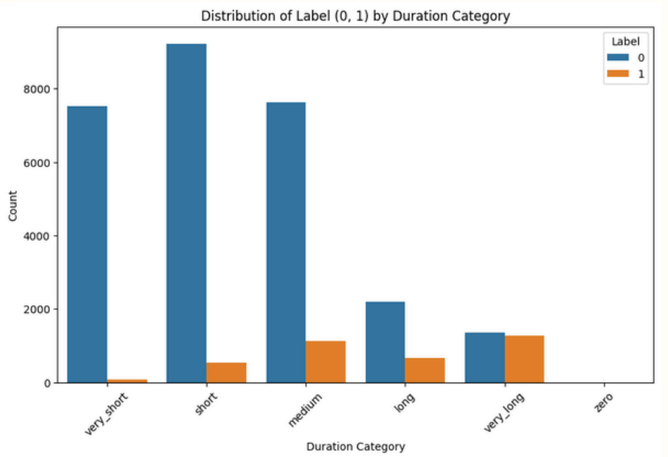
### poutcome

- 이전 참여 고객 중 재참여율 60% 이상 -> 충성 고객 관리가 중요




### duration

- 통화 시간이 길 수록 가입 확률이 높음
  가입 의사가 있는 고객이라면, 통화를 오래 할 것
  -> 이전에 통화시간이 길었던 고객에게 집중 마케팅




### day-month 병합

- day-month 병합으로 월말, 월초, 공휴일, 신년,  등 특별한 일자를 파악 가능
  -> 따른 마케팅 전략 설정이 용이해질 것

# 8.REFERENCES


데이터셋 및 캐글 관련 정보

* Kaggle - Bank Marketing Dataset
* UCI Machine Learning Repository - Bank Marketing

사용한 기법 및 알고리즘

* LGBM Documentation: LightGBM Official Documentation
* SHAP Library: SHAP GitHub Repository

데이터 불균형 처리

* A Guide to Handling Imbalanced Data

앙상블 방법론

* Ensemble Learning in ML

EDA 및 피처 엔지니어링 참고 자료

* Feature Engineering Guide
* Exploratory Data Analysis Techniques

# Q & A

# 감사합니다